



University
of Glasgow | School of
Computing Science

Client Selection in Federated Learning Environments: A Probabilistic Approach

Aquib Husain Iltija

School of Computing Science
Sir Alwyn Williams Building
University of Glasgow
G12 8QQ

A dissertation presented in part fulfilment of the requirements of the
Degree of Master of Science at The University of Glasgow

10 January 2025

Abstract

Federated Learning (FL) addresses critical challenges in data privacy and scalability for distributed machine learning. This thesis evaluates and enhances client selection strategies in FL to optimise model accuracy, communication efficiency, and fairness in heterogeneous environments. Using the Flower framework for simulations, the CIFAR-10 and MNIST datasets were used to implement and train a Convolutional Neural Network (CNN) in both IID and Non-IID scenarios.

This thesis introduces six client selection strategies, including the widely used random client selection method (FedAvg) as a baseline and novel approaches such as dynamic k and α selection. Each strategy was rigorously assessed on metrics including accuracy, loss, and communication rounds. Visualisations were employed to explore trade-offs, identifying configurations that optimise both model performance and resource usage.

This work advances FL by proposing probabilistic client selection strategies that enhance client participation dynamics, contributing to scalable and efficient FL solutions for real-world applications.

Education Use Consent

I hereby give my permission for this project to be shown to other University of Glasgow students and to be distributed in an electronic format.

Name: **Aquib Husain Iltija**

Signature: 

Acknowledgements

I am deeply grateful to my supervisor, Dr. Christos Anagnostopoulos, for introducing me to the fascinating domain of Federated Learning. His invaluable guidance, insightful feedback, and the additional resources and research papers he shared during our meetings greatly enhanced my understanding of the concepts and shaped the direction of this thesis.

My sincere gratitude also goes out to my family and friends for their constant encouragement and support along this journey.

Lastly, I would like to sincerely thank the University of Glasgow for providing the resources and a supportive academic environment that made this work possible.

Contents

1	Introduction	5
1.1	Aim and Objectives	5
2	Background and Related works	6
2.1	Federated Learning: Overview and Challenges	6
2.2	Foundation of Client Selection	7
2.3	Recent Advancements in Client Selection	7
2.4	Research Gaps and Motivation	8
3	Implementation	9
3.1	Workflow Overview	9
3.2	Tools and Infrastructure	10
3.3	Dataset and Pre-processing	11
3.3.1	Dataset Description	11
3.3.2	Data Distribution	11
3.3.3	Pre-processing Step	11
3.4	Model Architecture	11
3.5	Pre Setup for Federated Learning Simulation	12
3.6	Methods	13
3.6.1	Method 0: Centralised Training	13
3.6.2	Method 1: Federated Learning with Random Client Selection(FedAvg)	13
3.6.3	Method 2: Personalised Client Selection	14
3.6.4	Method 3: Dynamic Client Selection with Accuracy Threshold	15
3.6.5	Method 4: Client Selection based on Performance Comparison	16
3.6.6	Method 5: Top K Client Selection	17
3.6.7	Method 6: Dynamic Client Selection with K and Alpha (α)	18

4	Results and Discussions	20
4.1	Method 0: Centralised Training Results	20
4.2	Method 1: Random Client Selection Results	20
4.3	Method 2: Personalised Client Selection Results	20
4.4	Method 3: Dynamic Selection with Accuracy Threshold Results	21
4.5	Method 4: Performance-Based Client Selection Results	21
4.6	Method 5: Top-K Client Selection Results	21
4.7	Method 6: Dynamic K and Alpha Adjustment Results	21
4.8	Discussions	22
4.8.1	Dataset-Specific Reflections	22
4.8.2	General Observations and Reflections	22
4.8.3	Method Specific Reflections	22
4.8.4	Comparison of Methods	24
4.8.5	Insights from Visualisations	24
5	Conclusion and Future Works	27
5.1	Future Work	27
	Bibliography	29

Chapter 1: Introduction

The exponential growth of data generation across industries has driven significant advancements in machine learning (ML)[12]. However, this surge also raises critical concerns about data privacy, security, and efficiency in large scale data processing[16]. Traditional centralised ML approaches, which aggregate data in a central repository for model training, conflict with rigid privacy regulations and user concerns[16]. Federated Learning (FL), a decentralised approach, offers a solution by enabling multiple devices (e.g., smartphones, edge devices) to collaboratively train a shared model while storing data locally, this design not only protects user privacy but also reduces data transfer costs and enhances scalability in distributed environments[15].

FL has demonstrated its potential in real-world applications, such as personalised keyboard prediction systems (e.g., Gboard), privacy-preserving healthcare solutions, and scalable Internet of Things (IoT) networks[10]. However, despite these advantages, FL faces significant challenges, particularly in communication efficiency and client selection[13]. Communication efficiency refers to the frequent exchange of model updates between the central server and participating clients, which can be resource-intensive, especially in environments characterised by non-uniform data distributions or heterogeneous client capabilities[15].

Client selection, a fundamental component of FL, involves choosing a subset of available clients to participate in each training round[13]. Effective client selection can significantly impact the performance of the global model by optimising resource usage, ensuring lower communication costs, and accelerating convergence[13]. Traditional methods, such as random client selection strategies like FedAvg, often fail to account for critical factors such as client availability, computational resources, and data quality[15]. As a result, these methods may lead to imbalanced participation, slower convergence, and increased communication costs.

To address these challenges, this thesis focuses on developing and evaluating customised strategies for client selection in FL environments.

1.1 Aim and Objectives

This thesis aims to address the limitations of existing client selection strategies, as discussed in Section 2.4, by developing and evaluating probabilistic approaches that dynamically optimise client participation. By ranking clients based on performance metrics and introducing mechanisms such as adaptive thresholds and dynamic exclusion, the proposed methods seek to reduce communication costs, ensure fairness, and increase overall model performance.

The specific objectives of this thesis are:

Firstly, compare centralised and federated learning approaches to establish a baseline for performance evaluation. Building on this, the study implements and evaluates six methods for customised client selection, designed to dynamically optimise client participation. Lastly, this thesis aims to analyse and visualise the trade-offs between model accuracy and communication costs, identifying optimal configurations that balance performance and resource usage.

Chapter 2: Background and Related works

2.1 Federated Learning: Overview and Challenges

Federated Learning (FL) is a decentralised approach to machine learning that enables multiple devices (clients) to collaboratively train a global model without sharing raw data[15]. This paradigm enhances data privacy and security, making it particularly relevant in sensitive domains like healthcare and finance[16]. Instead of aggregating data at a central server, FL relies on locally computed model updates, which are sent to a central server for aggregation[15].

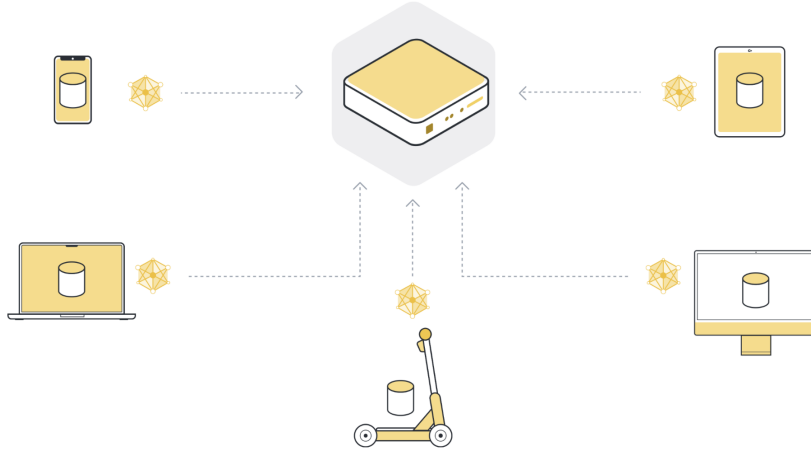


Figure 2.1: **Illustration of model aggregation in Federated Learning, showing updates from clients being combined at the server to refine the global model**[11].

While FL addresses data privacy concerns and promotes scalability, it introduces several key challenges:

Data Heterogeneity

The data across clients is often non-uniformly distributed. This variability, commonly referred to as Non-Independent and Identically Distributed (Non-IID) data, leads to inconsistencies in updates and slower convergence compared to uniformly distributed (IID) data[20].

Communication Bottlenecks

Frequent communication between clients and the central server imposes high bandwidth and computational demands, particularly for resource-constrained devices[10].

Client Diversity and Fairness

Clients vary in terms of computational resources, network capabilities, and data quality. Ensuring equitable participation across diverse clients remains an unresolved issue[13].

2.2 Foundation of Client Selection

Client selection is central to Federated Learning, as it determines which clients participate in training rounds, impacting model accuracy, communication efficiency, and fairness[13]. Traditional approaches have laid the groundwork for optimising this process:

FedAvg

A foundational algorithm where clients are selected randomly in each round. While simple and scalable, it fails to address data heterogeneity or resource constraints, leading to inefficiencies in Non-IID environments[15].

FedProx

Introduces a proximal term to mitigate the impact of data heterogeneity by encouraging local updates to remain closer to the global model. However, it does not adapt dynamically to client performance or resource conditions[14].

FedNova

Normalises updates to account for differences in local computations, ensuring fairness among clients. Despite its strengths, it overlooks dynamic prioritisation of clients based on real-time metrics[19].

FedOpt

Utilises adaptive optimisation techniques like Adam and Momentum to enhance convergence. However, it does not incorporate mechanisms to dynamically adjust client selection probabilities[17].

FedMA

Focuses on aligning and merging models in structurally heterogeneous environments. While effective for specific structural challenges, it lacks real-time adaptability for dynamic environments[18].

2.3 Recent Advancements in Client Selection

To address the limitations of traditional approaches, researchers have explored more adaptive and probabilistic methods for client selection:

FedPNS

Implements a probabilistic node selection strategy that prioritises underperforming clients based on their performance and data heterogeneity. However, it lacks the ability to adapt selection probabilities in real-time[20].

This approach, while improving efficiency and fairness, still falls short in several critical areas:

1. Convergence Rate:

Except for probabilistic node selection, it falls short in accelerating model convergence, resulting in an increased number of communication rounds and, consequently, higher costs[20].

2. Dynamic Adaptation:

It struggles to develop mechanisms that dynamically adjust node selection probabilities in real time, accounting for fluctuating client resources and network conditions[20].

2.4 Research Gaps and Motivation

Federated Learning (FL) has advanced significantly, but critical challenges remain unresolved, particularly in optimising both model accuracy and communication efficiency[13]. These challenges are magnified in heterogeneous environments, where variations in data distributions, computational capabilities, and network reliability introduce additional complexities [13].

Key Research Gaps

Static Client Selection Mechanisms: Existing strategies such as FedAvg (random client selection) and FedProx (addresses data heterogeneity) fail to adapt client participation probabilities in real time based on evolving performance metrics or resource constraints[15][14]. Similarly, methods like FedNova (normalising updates for fairness) and FedOpt (using adaptive optimisation) do not prioritise dynamic selection, often leading to underutilisation of high-performing clients or overburdening of low-performing ones, which delays model convergence [19][17].

Fairness in Non-IID Settings: Ensuring equitable participation across clients is particularly challenging in Non-IID scenarios, where imbalanced data distributions lead to biased model training, inconsistent convergence, and limited real-world utility[20]. For example, while FedMA (model alignment for structural heterogeneity) addresses specific structural challenges, it lacks the mechanisms to ensure fairness in data-imbalanced settings[18].

Lack of Real-Time Probabilistic Adaptation: Recent advancements like FedPNS (probabilistic node selection prioritising underperforming clients) improve efficiency and fairness but fall short in dynamically adjusting selection probabilities in real time, particularly under fluctuating client resources and network conditions[20].

Addressing these gaps is critical for improving the scalability, robustness, and real-world effectiveness of Federated Learning systems. The challenges outlined in this section are directly addressed through the methodologies proposed in Section 3.6.

Chapter 3: Implementation

This chapter outlines the implementation of Federated Learning (FL) using the Flower (FLWR) framework to evaluate client selection strategies on CIFAR-10 and MNIST datasets[1][2]. The focus is on optimising performance under IID and non-IID distributions while minimising communication rounds(interaction between server and clients) to enhance efficiency and reduce costs.

3.1 Workflow Overview

To give a high-level view of the implementation, the process is summarized in the following steps:

Dataset Preparation: The **CIFAR-10** and **MNIST** datasets, commonly used for image classification tasks, were chosen for this study. CIFAR-10 offers diverse image categories, making it ideal for evaluating complex models, while MNIST provides a simpler benchmark for comparison[2]. These datasets were distributed among clients in both IID (Independent and Identically Distributed) and Non-IID settings to simulate real-world data variability and heterogeneity, as detailed in Section 3.3.

Pre-processing: To ensure the datasets are compatible with the model and to enhance their generalisation capabilities, **normalisation** and **augmentation** techniques were applied. Normalisation scaled pixel values to a standard range, while transformations such as random cropping and horizontal flipping (for CIFAR-10) improved the diversity of training data and reduced overfitting risks.

Model Architecture: A **Convolutional Neural Network (CNN)** was employed as the model for this implementation. CNNs are highly effective in extracting spatial and hierarchical features from images, making them suitable for tasks like image classification. The architecture was tailored with convolutional layers for feature extraction, pooling layers for dimensionality reduction, and fully connected layers for classification[3].

Federated Learning Framework: The **Flower (FLWR)** framework was used to simulate a federated learning environment, where a centralised server interacts with decentralised clients to train the model[1]. This framework facilitated the implementation of various client selection strategies and the aggregation of model updates, ensuring scalability and adaptability for experimentation.

Evaluation: The performance of the implemented strategies was measured using key metrics such as **loss**, **accuracy**, and **total communication rounds** (for specific strategies). Accuracy quantified the predictive performance of the model, loss provided insights into optimisation and convergence, and total communication rounds assessed the efficiency of client-server interactions. These metrics were crucial in evaluating trade-offs between model quality, resource usage, and communication efficiency.

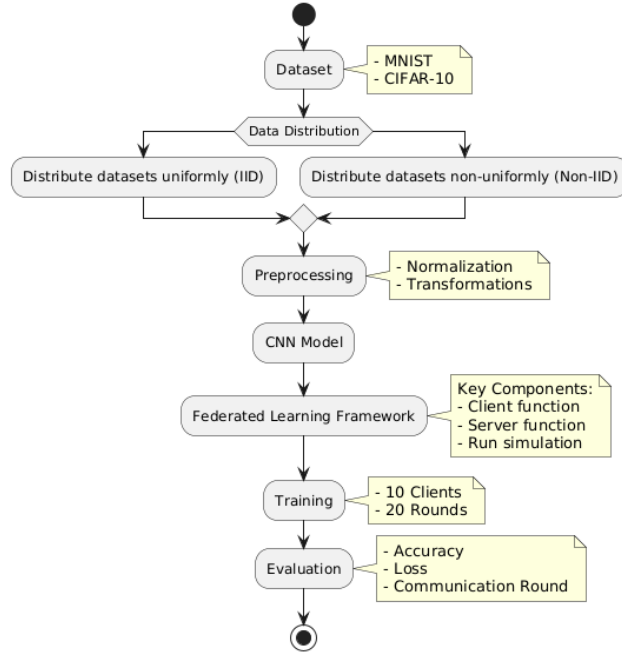


Figure 3.1: **Workflow of Federated Learning implementation, from data pre-processing to model evaluation.**

3.2 Tools and Infrastructure

The implementation was conducted using the following tools and frameworks:

Google Colab: A cloud-based platform that provides a Python environment with free GPU access, making it suitable for resource-intensive simulations[4]. It was chosen for its ease of use, accessibility, and seamless integration with various Python libraries, enabling efficient implementation and testing of federated learning strategies.

Flower (FLWR): A federated learning framework used to simulate client-server communication and implement custom client selection strategies. FLWR’s flexibility and scalability made it ideal for designing dynamic and personalised methods while supporting real-world FL scenarios[1].

PyTorch: A deep learning library used to design and train neural networks[5]. Its dynamic computation graph and user-friendly API were critical in building the Convolutional Neural Network (CNN) models for the CIFAR-10 and MNIST datasets[2].

Torchvision: A PyTorch companion library that streamlined dataset handling and transformations [6]. It was used to load the datasets and apply essential pre-processing steps, including normalisation and augmentation, to improve model generalisation.

NumPy: A library for numerical operations and data manipulation[7]. It played a key role in pre-processing, such as calculating client selection probabilities and managing dataset partitions efficiently.

Matplotlib: A visualisation library used to plot results[8]. It provided clear and interpretable visualisations of key metrics, such as accuracy, loss, and communication rounds, helping to analyse the performance of different strategies.

These tools were selected for their compatibility with Google Colab and their ability to support computationally intensive tasks. Together, they provided a robust framework for developing and evaluating federated learning methods efficiently.

3.3 Dataset and Pre-processing

3.3.1 Dataset Description

The implementation employs two datasets:

CIFAR-10: Comprising 60,000 (**32×32 RGB**) images across 10 classes, split into 50,000 training and 10,000 testing samples. It serves as a moderately complex benchmark[2].

MNIST: Containing 70,000 grayscale images (**28×28 pixels**) of handwritten digits, split into 60,000 training and 10,000 testing samples. It serves as a simpler benchmark[2].

Both the dataset was divided into training, validation, and testing splits, with local datasets allocated as **80% training** and **20% validation**[1].

3.3.2 Data Distribution

Data was distributed among 10 clients to simulate federated settings[1]:

IID (Independent and Identically Distributed): Each client receives an equal and representative portion of all classes.

Non-IID (Non-Independent and Non-Identically Distributed): Uneven distribution mimics real-world scenarios. For both datasets, two clients receive images from classes 1, 2, and 3. Two clients receive images from classes 4, 5, and 6. Two clients receive images from classes 7, 8, 9, and 10. Four clients receive images from all 10 classes but with different label distributions. The test set includes all classes equally distributed.

3.3.3 Pre-processing Step

Normalisation: The CIFAR-10 dataset was normalised using a mean of [**0.5, 0.5, 0.5**] and a standard deviation of [**0.5, 0.5, 0.5**], while the MNIST dataset used a mean of **0.1307** and a standard deviation of **0.3081**. Normalisation adjusted pixel values to a standard scale, ensuring consistent input across datasets and improving model convergence during training[1][2].

Transformations: Data augmentation techniques, such as random cropping and horizontal flipping, were applied to the CIFAR-10 dataset. These transformations introduced variability in the training data, enhancing the model’s ability to generalise and reducing overfitting, especially for complex datasets like CIFAR-10[1][2].

Batching: Data was loaded using PyTorch’s **DataLoader** with a batch size of **32**. Batching improved computational efficiency by allowing parallel processing of multiple samples during each iteration, reducing overall training time and ensuring efficient memory usage[1].

3.4 Model Architecture

A Convolutional Neural Network (CNN) was implemented to handle both datasets, leveraging its ability to extract and classify features from images effectively[3][1]. The network’s structure included:

Convolutional Layers: Two convolutional layers were used for feature extraction. These layers applied filters to identify patterns such as edges and textures in the input images, enabling the model to learn hierarchical representations of the data.

Input Channel: The input layer was configured with **3 channels for CIFAR-10 (RGB images)** and **1 channel for MNIST (grayscale images)**. This setup ensured compatibility with the colour depth of each dataset[2].

Pooling Layers: Max pooling was applied after each convolutional layer to downsample the feature maps, reducing their spatial dimensions while retaining critical features. This step helped lower computational complexity and prevent overfitting.

Fully Connected Layers: Three fully connected layers followed the feature extraction process. The extracted features were flattened and passed through these layers to map them to the output classes, enabling accurate classification.

Activation Function: ReLU (Rectified Linear Unit) was used as the activation function in the network to introduce non-linearity[9]. This allowed the model to learn complex relationships between features.

Output Layer: The final layer produced class probabilities for 10 classes, corresponding to the labels in both datasets. This enabled the model to predict the category of each input image.

3.5 Pre Setup for Federated Learning Simulation

The FL simulation was implemented using the Flower framework, a versatile tool for simulating client-server communication, enabling local training on client devices and global model updates on the server[1]. The setup comprised the following components:

Client Setup: Helper functions were defined to retrieve and update model parameters, ensuring seamless communication between clients and the server. A custom **FlowerClient** class was created to handle local training, evaluation, and parameter updates. This allowed each client to independently process its data while contributing to the global model[1].

Server Setup: The server was configured with strategies such as **FedAvg** or custom client selection strategies. These strategies dictated how client updates were aggregated to improve the global model while balancing factors like accuracy, communication efficiency, and fairness[1].

Simulation Execution: Simulations were executed using the **run simulation** function, which facilitated interactions between the server and clients. Backend configurations specified client resources, such as CPU or GPU, ensuring efficient utilisation for training and evaluation[1].

This foundational setup remained consistent across all methods, with variations introduced only in the client selection strategies to evaluate their impact on performance and communication efficiency.

3.6 Methods

3.6.1 Method 0: Centralised Training

This method implements a traditional centralised training approach to establish a baseline for evaluating the performance of Federated Learning (FL) strategies[1]. The entire dataset was utilised without partitioning, ensuring all data was available for training in a single node.

The pre-setup and model architecture mentioned in Section 3.5 and Section 3.4 respectively, were utilised for this method. The key implementation details are as follows:

Datasets: Both **MNIST** and **CIFAR-10** datasets were used to evaluate performance under varying levels of data complexity[2].

Training Process: The model was trained for **20 epochs** using the Adam optimiser and Cross-Entropy Loss function. Accuracy and loss metrics were recorded at each epoch to monitor model convergence for both datasets.

Testing: Final testing was conducted on separate test sets for MNIST and CIFAR-10 to evaluate the model’s generalisation performance.

Algorithm 1 Centralised Training (Method 0)

Server executes:

- 1: Initialize model parameters W
- 2: **for** each epoch $e = 1, 2, \dots, E$ **do**
- 3: Divide dataset D into batches
- 4: **for** each batch $b \in D$ **do**
- 5: Perform forward pass and compute loss $\mathcal{L}$
- 6: Perform backward pass to compute gradients
- 7: Update W using gradient descent:

$$W \leftarrow W - \eta \nabla \mathcal{L}(W)$$

- 8: **end for**
 - 9: **end for**
 - 10: Evaluate W on the test dataset
 - 11: Return final model W
-

Summary: Centralised training serves as a benchmark for comparing the distributed approaches in subsequent methods. This method provides an upper bound for model performance under ideal conditions, with no constraints on data distribution or communication overhead. The results of centralized training, used as a baseline for comparison, are discussed in Section 4.1.

3.6.2 Method 1: Federated Learning with Random Client Selection(FedAvg)

This method explores the foundational **Federated Averaging (FedAvg)** algorithm, where a random selection of clients participates in each training round[15]. This method serves as the baseline for comparing advanced client selection strategies.

The pre-setup for FL and model architecture mentioned earlier were utilised for this method. Both MNIST and CIFAR-10 datasets were used, enabling evaluation across varying data complexities. The key implementation details are as follows:

Client-Server Interaction: A total of **10 clients** were simulated, each receiving an equal portion of the dataset in IID and Non-IID distributions. Clients trained locally for **1 epoch per round**, with their updates aggregated by the server using the FedAvg strategy. All clients participated in training in the default configuration Fraction fit = 1.0, while only 50% participated in evaluation.

Varying Client Participation: To analyse the impact of client participation, additional experiments were conducted by modifying the participation rates: **50% Participation:** Fraction fit = 0.5. **70% Participation:** Fraction fit = 0.7. **20% Participation:** Fraction fit = 0.2. These configurations were applied to both datasets for comparative analysis.

Algorithm 2 Federated Averaging (Method 1)

Server executes:

- 1: Initialize global model parameters W
- 2: **for** each round $r = 1, 2, \dots, R$ **do**
- 3: Randomly select k clients
- 4: Send W to selected clients
- 5: Receive updates W_c from clients
- 6: Aggregate updates:

$$W \leftarrow \frac{\sum_{c=1}^k n_c W_c}{\sum_{c=1}^k n_c}$$

where n_c is the number of samples for client c

- 7: **end for**
- 8: Return final model W

ClientUpdate(W):

- 1: **for** each local epoch e **do**
 - 2: Train on local data
 - 3: **end for**
 - 4: Return updated parameters W_c to server
-

Summary: FedAvg provided a stable benchmark for evaluating client selection strategies. While higher participation rates improved convergence, they introduced greater communication overhead. This method serves as a reliable foundation for exploring personalised and dynamic strategies in subsequent methods. The performance of this method under varying participation rates is presented in Section 4.2.

3.6.3 Method 2: Personalised Client Selection

In this method, a personalised client selection strategy was implemented to prioritise under-performing clients. By dynamically adjusting client selection probabilities based on their previous accuracy, this approach aimed to improve the convergence of the global model. The experiments were conducted on both **CIFAR-10** and **MNIST** datasets under **IID** and **Non-IID** data distributions.

The pre-setup for FL and model architecture mentioned earlier in Section 3.5 and Section 3.4 respectively, were utilised for this method. The key implementation details are as follows:

Personalised Selection Strategy: The client selection probability was dynamically adjusted using the formula:

$$p[k, t + 1] = 1 - a[k, t]$$

where $a[k, t]$ represents the accuracy of client k at round t . This ensures that clients with lower accuracies are assigned higher probabilities for selection, allowing the model to focus on improving weaker-performing nodes.

Training: A total of **10 clients** participated in the simulation. For the first round, all clients were selected to establish a baseline. In subsequent rounds, a maximum of **5 clients** were selected based on the personalised probabilities.

Algorithm 3 Personalised Client Selection (Method 2)

Server executes:

- 1: Initialize global model parameters W
- 2: **for** each round $r = 1, 2, \dots, R$ **do**
- 3: Compute selection probability for each client:

$$P_c = 1 - A_c$$

where A_c is the accuracy of client c

- 4: Select k clients based on P_c
 - 5: Send W to selected clients
 - 6: Receive updates W_c from clients
 - 7: Aggregate updates: $W \leftarrow FedAvg(W_c)$
 - 8: **end for**
 - 9: Return final model W
-

Testing: Each selected client trained locally for **1 epoch per round**, for total 20 rounds, and the server aggregated their updates. Global model performance was evaluated using metrics such as accuracy and loss.

Summary: This method demonstrated the potential of personalised client selection in Federated Learning. By prioritising weaker clients, it achieved improved convergence on both datasets compared to random selection. The **Non-IID distribution** posed additional challenges, highlighting the importance of adaptive strategies in heterogeneous environments.

3.6.4 Method 3: Dynamic Client Selection with Accuracy Threshold

This method introduces a threshold-based client selection strategy, aimed at focusing training resources on weaker-performing clients by excluding those whose accuracy exceeds a pre-defined threshold. The method was applied to both **CIFAR-10** and **MNIST** datasets under **IID** and **Non-IID** data distributions.

The pre-setup for FL and model architecture mentioned earlier in Section 3.5 and Section 3.4 respectively, were utilised for this method. The key implementation details are as follows:

Threshold-Based Client Exclusion: Clients with accuracy greater than or equal to a pre-defined threshold of **0.9** were excluded from training in subsequent rounds. Additionally, clients with an accuracy of **1.0** were excluded to avoid overfitting on highly accurate clients.

Selection Probability: Remaining clients were assigned selection probabilities using the formula:

$$p[k, t + 1] = 1 - a[k, t]$$

as discussed in Subsection 3.6.3. This ensured that clients with lower accuracy had a higher likelihood of being selected.

Training: In the first round, all clients participated in training to establish baseline performance. In subsequent rounds, clients were dynamically selected based on their accuracy relative to the threshold and probabilistic criteria.

Algorithm 4 Threshold Based Client Exclusion (Method 3)

Server executes:

- 1: Initialize global model parameters W
- 2: **for** each round $r = 1, 2, \dots, R$ **do**
- 3: Compute selection probability for each client:

$$P_c = 1 - A_c$$

- 4: Exclude clients with $A_c \geq threshold$
 - 5: Select remaining clients based on P_c
 - 6: Send W to selected clients
 - 7: Receive updates W_c from clients
 - 8: Aggregate updates: $W \leftarrow FedAvg(W_c)$
 - 9: **end for**
 - 10: Return final model W
-

Metrics such as global accuracy, communication rounds, and the effectiveness of the exclusion threshold were recorded.

Summary: The threshold-based selection strategy effectively focused training efforts on weaker-performing clients, improving model convergence for CIFAR-10 under Non-IID settings. By excluding highly accurate clients, the method ensured efficient use of training resources. The results highlight the potential of dynamic exclusion mechanisms in Federated Learning.

3.6.5 Method 4: Client Selection based on Performance Comparison

In this method, a performance-based client selection strategy was implemented, focusing on clients whose accuracy did not improve compared to their performance in the previous round. By prioritising "underperforming" clients for training, this method aims to enhance global model convergence while ensuring efficient resource utilisation. The method was applied to both **CIFAR-10** and **MNIST** datasets in **IID** and **Non-IID** settings.

The pre-setup for FL and model architecture mentioned earlier in Section 3.5 and Section 3.4 respectively, were utilised for this method. The key implementation details are as follows:

Performance Comparison Logic: Clients were identified as underperforming if their **current accuracy** was less than or equal to their **previous accuracy**. Underperforming clients were prioritised for training in the subsequent round. If all clients demonstrated performance improvement, training was terminated early.

Dynamic Participation: In the first round, all clients participated in training to establish baseline accuracies. From the second round onwards, clients were selected dynamically based on their relative performance.

Stopping Criterion: Training was terminated when all clients showed improved performance in the previous round, avoiding unnecessary computation.

Algorithm 5 Accuracy Comparison-Based Selection (Method 4)

Server executes:

- 1: Initialize global model parameters W
 - 2: **for** each round $r = 1, 2, \dots, R$ **do**
 - 3: Compare current accuracy A_c with previous accuracy A_c^{prev}
 - 4: Select clients with $A_c \leq A_c^{prev}$
 - 5: **if** no clients are selected **then**
 - 6: Randomly select k clients
 - 7: **end if**
 - 8: Send W to selected clients
 - 9: Receive updates W_c from clients
 - 10: Aggregate updates: $W \leftarrow FedAvg(W_c)$
 - 11: **end for**
 - 12: Return final model W
-

Summary: This method effectively identified and prioritised underperforming clients, focusing training resources on areas requiring improvement. The performance-based selection strategy demonstrated the potential to accelerate model convergence while reducing overall communication rounds in scenarios where clients reached consistent performance quickly.

3.6.6 Method 5: Top K Client Selection

This method introduces a fixed selection of **K clients** in each training round. The method is designed to either prioritise underperforming clients or ensure diversity through random selection, aiming to strike a balance between focused training and global model improvement. This approach was applied to both **CIFAR-10** and **MNIST** datasets under **IID** and **Non-IID** data distributions.

The pre-setup for FL and model architecture mentioned earlier in Section 3.5 and Section 3.4 respectively, were utilised for this method. The key implementation details are as follows:

Client Selection Strategy: For first round all clients participated to establish initial accuracy metrics. Subsequent Rounds, By default, **K=3** clients were selected per round. Two approaches were explored:

Random Selection: If no specific selection criteria were met, K clients were randomly selected.

Lowest Accuracy Selection: Alternatively, the top-K clients with the lowest accuracies were selected.

Flexibility in Selection: The method dynamically handled scenarios where fewer than K clients met the underperformance criteria by switching to random selection. This ensured consistent participation and avoided stagnation in training.

Training proceeded for 20 rounds, ensuring ample evaluation of client performance across diverse setups.

Algorithm 6 Accuracy Comparison with K Clients (Method 5)

Server executes:

- 1: Initialize global model parameters W
 - 2: **for** each round $r = 1, 2, \dots, R$ **do**
 - 3: Compare current accuracy A_c with previous accuracy A_c^{prev}
 - 4: Select clients with $A_c \leq A_c^{prev}$
 - 5: **if** no clients are selected **then**
 - 6: Randomly select k clients
 - 7: **end if**
 - 8: Send W to selected clients
 - 9: Receive updates W_c from clients
 - 10: Aggregate updates: $W \leftarrow FedAvg(W_c)$
 - 11: **end for**
 - 12: Return final model W
-

Summary: The fixed Top K Selection strategy effectively combined focus and diversity in training by leveraging two distinct selection mechanisms. This approach provided insights into the trade-offs between prioritising underperforming clients and maintaining client diversity, showcasing the adaptability of federated learning frameworks to varying real-world scenarios.

3.6.7 Method 6: Dynamic Client Selection with K and Alpha (α)

This method introduces a **dynamic client selection strategy** by incorporating two parameters:

K (number of clients): Determines how many clients to select in each round.

Alpha α (smoothing factor): Adjusts the probabilistic selection of clients based on their performance.

The approach balances exploration and exploitation by ensuring weaker clients participate frequently while maintaining diversity. This was applied to **CIFAR-10** and **MNIST** datasets under **IID** and **Non-IID** distributions.

The pre-setup for FL and model architecture mentioned earlier in Section 3.5 and Section 3.4 respectively, were utilised for this method. The key implementation details are as follows:

Initial Rounds (Exploration): K least accurate clients were selected to ensure that all clients participated at least once.

Subsequent Rounds (Exploitation): Probabilities for client selection were calculated using the formula:

$$p[k, t + 1] = 1 - \alpha * a[k, t]$$

The strategy favoured underperforming clients without excluding better-performing ones entirely.

Parameters: Experiments were conducted for different combinations of K and alpha(α) values: K: {2, 3, 5, 7} , α : {0.1, 0.2, 0.5, 0.8}

Training proceeded for 20 rounds, accuracy and communication rounds were noted.

Algorithm 7 Dynamic K and α Selection (Method 6)

Server executes:

- 1: Initialize global model parameters W
- 2: Set k : Number of clients, α : Smoothing factor
- 3: **for** each round $r = 1, 2, \dots, R$ **do**
- 4: **if** $r \leq I$ **then**
- 5: Select k least accurate clients
- 6: **else**
- 7: **for** each client c **do**
- 8: Compute selection probability:

$$P_c = \max(1 - \alpha \cdot A_c, 0)$$

- 9: **end for**
 - 10: Select k clients based on P_c
 - 11: **end if**
 - 12: Send W to selected clients
 - 13: Receive updates W_c from clients
 - 14: Aggregate updates: $W \leftarrow FedAvg(W_c)$
 - 15: **end for**
 - 16: Return final model W
-

Visualisation: Scatter plots were generated to analyse the relationship between final accuracy and total communication rounds for varying K and α values. Results were saved as interactive visualisations for deeper insights.

Summary: The **Dynamic K and Alpha Selection** strategy demonstrated the flexibility of parameter tuning in federated learning. By balancing client participation, the method addressed trade-offs between accuracy and communication overhead effectively. Results highlighted the importance of selecting appropriate K and α values based on the dataset and distribution characteristics. This method provided insights into the adaptability of federated systems in varying scenarios.

Chapter 4: Results and Discussions

This chapter presents the results of Federated Learning (FL) using the Flower (FLWR) framework on CIFAR-10 and MNIST datasets. It evaluates model performance, communication efficiency, and client contribution across all methods discussed in the implementation chapter, considering various client selection strategies for both IID and Non-IID data distributions[2].

4.1 Method 0: Centralised Training Results

Table 4.1 shows the results of centralised training, providing a baseline for performance in terms of loss and accuracy.

Dataset	Loss	Accuracy
CIFAR10	0.05396	0.47
MNIST	0.00494	0.97

Table 4.1: Results of centralised training (Method 0)

4.2 Method 1: Random Client Selection Results

The outcomes of random client selection under different participation rates are summarized in Table 4.2 and Table 4.3, highlighting the variations in accuracy and loss.

CIFAR10:

Client Participation (%)	Loss	Accuracy
100	0.039	0.55
70	0.0398	0.55
50	0.0390	0.56
20	0.0414	0.54

Table 4.2: Results of varying client participation for CIFAR-10 (Method 1)

MNIST:

Client Participation (%)	Loss	Accuracy
100	0.0013	0.98
70	0.0013	0.98
50	0.0010	0.98
20	0.0010	0.98

Table 4.3: Results of varying client participation for MNIST (Method 1)

4.3 Method 2: Personalised Client Selection Results

Results for personalised client selection are shown in Table 4.4, where clients were prioritised based on formula discussed in Subsection 3.6.3, with a maximum of 5 clients selected per round after the first.

Dataset	Distribution	Initial Accuracy (%) Round 1	Final Accuracy (%) Round 20
CIFAR10	IID	0.28	0.55
CIFAR10	NON-IID	0.20	0.40
MNIST	IID	0.91	0.98
MNIST	NON-IID	0.28	0.96

Table 4.4: **Results of personalised client selection training (Method 2)**

4.4 Method 3: Dynamic Selection with Accuracy Threshold Results

The Table 4.5 presents results for accuracy threshold-based selection under IID conditions, excluding clients with an accuracy greater than 0.9 from further rounds.

Dataset	Distribution	Initial Accuracy (%)	Final Accuracy (%)	Early Stopping
CIFAR10	IID	0.25	0.54	ROUND 8
MNIST	IID	0.91	0.91	ROUND 2

Table 4.5: **Results of threshold based client selection training (Method 3)**

4.5 Method 4: Performance-Based Client Selection Results

The Table 4.6 below highlights results for performance comparison-based selection, where clients with no accuracy improvement over previous rounds were prioritised.

Dataset	Distribution	Initial Accuracy (%)	Final Accuracy (%)	Early Stopping
CIFAR10	IID	0.29	0.29	ROUND 2
MNIST	IID	0.90	0.90	ROUND 2

Table 4.6: **Results of performance comparison based client selection training (Method 4)**

4.6 Method 5: Top-K Client Selection Results

Table 4.7 presents results for Top K client selection (K=3), prioritising either random clients or those with the lowest accuracy in each round

Dataset	Distribution	Initial Accuracy (%)	Final Accuracy (%)	Early Stopping
CIFAR10	IID	0.31	0.54	ROUND 3
MNIST	IID	0.92	0.98	ROUND 2

Table 4.7: **Results of Top K client client selection training (Method 5)**

4.7 Method 6: Dynamic K and Alpha Adjustment Results

Results for dynamic client selection are shown in Table 4.8, using parameters K=3 and varying α values (e.g., 0.1, 0.2, 0.5) to balance participation diversity and focus.

Dataset	Distribution	Alpha (α)	Initial Accuracy (%)	Final Accuracy (%)
CIFAR10	IID	0.5	0.30	0.55
CIFAR10	IID	0.2	0.31	0.54
CIFAR10	IID	0.1	0.29	0.57
CIFAR10	NON-IID	0.5	0.11	0.44
CIFAR10	NON-IID	0.2	0.20	0.39
CIFAR10	NON-IID	0.1	0.19	0.45
MNIST	IID	0.5	0.89	0.98
MNIST	IID	0.2	0.91	0.98
MNIST	IID	0.1	0.90	0.98
MNIST	NON-IID	0.5	0.30	0.36
MNIST	NON-IID	0.2	0.30	0.95
MNIST	NON-IID	0.1	0.30	0.88

Table 4.8: **Results of dynamic client selection with $K = 3$ and varying alpha values (Method 6)**

4.8 Discussions

This section reflects on the results obtained, offering an interpretation of the observed trends and their implications in the context of federated learning and client selection strategies.

4.8.1 Dataset-Specific Reflections

The performance variations across MNIST and CIFAR-10 datasets reveal important insights into the effectiveness of the proposed methods. MNIST, with its simpler structure, consistently showed faster convergence and higher accuracy across all methods. CIFAR-10, on the other hand, presented greater challenges due to its complexity, particularly in non-IID distributions. The dynamic methods demonstrated adaptability, but non-IID settings highlighted the importance of balancing client diversity with accuracy improvements, as uneven data distributions often delay performance.

4.8.2 General Observations and Reflections

Across the methods, a balance between accuracy and communication efficiency emerged as a critical challenge. While higher client participation improved accuracy, it also increased communication between server and clients, making it less feasible for resource-constrained environments. Dynamic strategies like personalised selection and threshold-based exclusion offered better efficiency, especially in targeting underperforming clients, though they required careful tuning to maintain fairness.

4.8.3 Method Specific Reflections

Centralised Training (Method 0): This method was implemented to establish a baseline for comparison with federated learning approaches. It provided an upper limit for model accuracy by utilising the entire dataset on a single node, demonstrating the potential of centralised training under ideal conditions. While effective as a benchmark, it lacks real-world feasibility due to privacy concerns and communication constraints.

Random Client Selection (FedAvg - Method 1): This method was used as an introductory federated learning approach to compare its performance with centralised training. It also explored the impact of varying client participation rates on accuracy. For CIFAR-10, good accuracy was observed when 50% of clients participated, possibly due to a balance

between client diversity and model updates. For MNIST, being a simpler dataset, accuracy remained consistent across varying client participation rates. This highlights the robustness of federated learning with simpler data but indicates that more complex datasets benefit from optimised client participation strategies.

Personalised Client Selection (Method 2): This method prioritised weaker clients by assigning probabilities inversely proportional to their accuracy $p[k, t + 1] = 1 - a[k, t]$. In the initial round, all clients participated to establish a baseline. Subsequently, clients were selected based on their assigned probabilities, ensuring that underperforming clients were more likely to participate in training rounds. By round 15, almost all clients had participated, showcasing the method’s inclusivity over multiple rounds. However, **this approach struggled to achieve high accuracy under non-IID distributions**, highlighting challenges in ensuring fairness and diversity in client contributions across heterogeneous data distributions.

Dynamic Client Selection with Accuracy Threshold (Method 3): In this method, an accuracy threshold of **0.9** was introduced to exclude high-performing clients from further training rounds, ensuring that all participating clients contributed meaningfully to the global model. For CIFAR-10, all clients reached the threshold by round 8, whereas for MNIST, this occurred much earlier, by round 2, due to its simpler nature. Once the threshold was met, **early stopping was observed**, with subsequent rounds showing neutral results as accuracy neither improved nor decreased. Due to the early stopping behaviour, this method was not performed for non-IID distributions, as it would limit the evaluation of its effectiveness in such scenarios. This highlights the need for diverse methods to make progress in the subsequent rounds to eliminate early stopping.

Performance Comparison-Based Selection (Method 4): This method stored the accuracies of each client after the first round and compared their current accuracies with those from the previous round. In the initial round, all clients participated to establish a baseline. For subsequent rounds, clients whose accuracy did not improve were prioritised for selection. However, **this method failed to perform effectively, as early stopping was observed by round 2** for both CIFAR-10 and MNIST. This occurred because most clients showed slight accuracy improvements after the first round, leaving no clients to prioritise under the selection criteria. This method was not implemented for NON-IID because of the early stopping and predictive results. Consequently, this method underperformed significantly compared to others.

Top-K Client Selection (Method 5): To address the issue of early stopping observed in previous methods, this method introduced a fixed value of **K = 3** to ensure continued training beyond the threshold or accuracy stagnation. Initially, accuracy threshold and performance comparison strategies were implemented, but upon encountering early stopping, K random clients were selected for participation in subsequent rounds. **This approach allowed the training to continue up to 20 rounds, resulting in slight improvements in overall accuracy.** The method demonstrated noticeable improvements for both CIFAR-10 and MNIST datasets, effectively mitigating the early stopping issue and maintaining training momentum.

Dynamic K and Alpha Selection (Method 6): Building on the improvements from Method 5, this method combined K and alpha (α) for a dynamic client selection strategy. For the initial rounds (up to round 5), K random clients were selected as a warmup phase to ensure all clients participated in early training. After round 5, client selection was based on a probabilistic formula: $p[k, t + 1] = 1 - \alpha * a[k, t]$ as earlier discussed in subsection 3.6.7

where α acted as a smoothing factor to adjust the influence of accuracy on selection probabilities. This approach allowed weaker clients to have higher probabilities of being selected, maintaining diversity and improving convergence.

This method showcased the highest accuracy for both CIFAR-10 and MNIST, outperforming all previous strategies. It was also evaluated under non-IID conditions, where it demonstrated resilience by balancing fairness and performance. Experimentation with various K values (K = 2,3,5,7) and alpha values ($\alpha = 0.1, 0.2, 0.5, 0.8$), revealed that certain combinations optimised both accuracy and communication efficiency. The results were visualised, showing the trade-offs between accuracy and communication rounds, further validating the method’s adaptability and effectiveness in federated learning environments.

4.8.4 Comparison of Methods

As discussed in Section 3.6.7, **Dynamic K and Alpha Selection (Method 6)** stood out as the most effective and adaptable, achieving the highest accuracy and communication efficiency across both IID and non-IID distributions. Its dynamic adjustment of client participation based on K and alpha values allowed for optimal resource allocation while maintaining fairness and diversity, making it the best-performing method overall. **Top K Client Selection (Method 5)** also demonstrated notable improvements by addressing early stopping through fixed K value selections. While it maintained training momentum and improved accuracy compared to earlier strategies, its fixed K value limited flexibility and adaptability in diverse scenarios.

Personalised Client Selection (Method 2) achieved significant improvements by prioritising weaker clients, ensuring inclusivity and gradual convergence. However, its reliance on careful parameter tuning and reduced performance under non-IID distributions highlighted the need for further refinement. **Threshold-Based Exclusion (Method 3)** optimised resources by focusing on underperforming clients but suffered from early stopping and reduced client diversity, particularly in simpler datasets like MNIST, making it unsuitable for non-IID conditions. In contrast, **Performance Comparison-Based Selection (Method 4)** underperformed due to early stopping as most clients showed minor accuracy improvements after the initial round, failing to demonstrate consistent gains.

Overall, the results highlight the trade-offs between static and dynamic strategies in federated learning. Dynamic strategies, particularly those incorporating probabilistic elements like K and alpha were more effective in addressing challenges such as early stopping, resource optimisation, and fairness, showcasing their potential for real-world federated learning applications. These comparisons fulfill the objectives mentioned in Section 1.2, specifically optimising model performance while minimising resource usage.

4.8.5 Insights from Visualisations

The visualisations of accuracy versus communication rounds provide valuable insights into identifying efficient client configurations. Each point on the graph represents a combination of K and α values, with the legends indicating their specific configurations. Among these, the highlighted points stand out as ideal configurations, balancing high accuracy with minimal communication round.

For instance, in the CIFAR-10 non-IID visualisation, the highlighted point corresponds to a configuration with optimal K and α values, achieving superior accuracy while requiring fewer communication rounds compared to other combinations. Similarly, for MNIST, the highlighted point demonstrates how even in simpler datasets, thoughtful selection of client

parameters can yield efficient training outcomes. Notably, this method's adaptability under both IID and non-IID conditions is evident in the consistent patterns of efficiency and performance across different datasets.

These visualisations not only validate the effectiveness of dynamic selection methods like Dynamic K and Alpha Selection (Method 6) but also highlight their practicality for real-world federated learning setups. The ability to visualise and select configurations ensures that data-driven decisions can be made, maximizing both performance and communication efficiency.

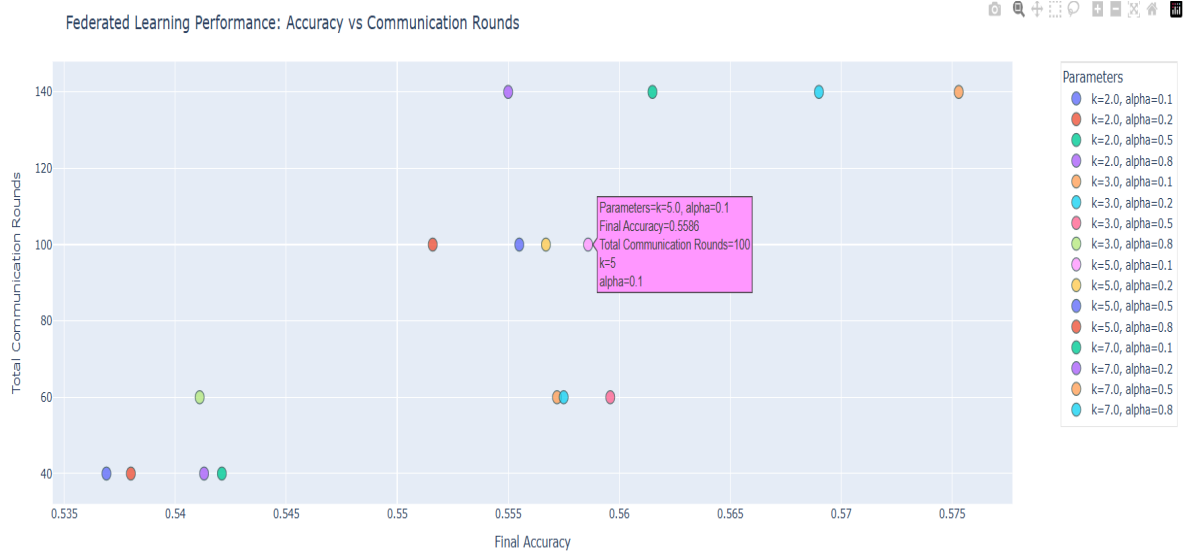


Figure 4.1: Accuracy versus communication rounds for CIFAR-10 under IID conditions, with the highlighted point representing the best accuracy achieved with minimal communication rounds.



Figure 4.2: Accuracy versus communication rounds for CIFAR-10 under Non-IID conditions, showing the highlighted point as the optimal configuration balancing accuracy and communication efficiency.



Figure 4.3: Accuracy versus communication rounds for MNIST under IID conditions, with the highlighted point indicating the best accuracy with fewer communication rounds.

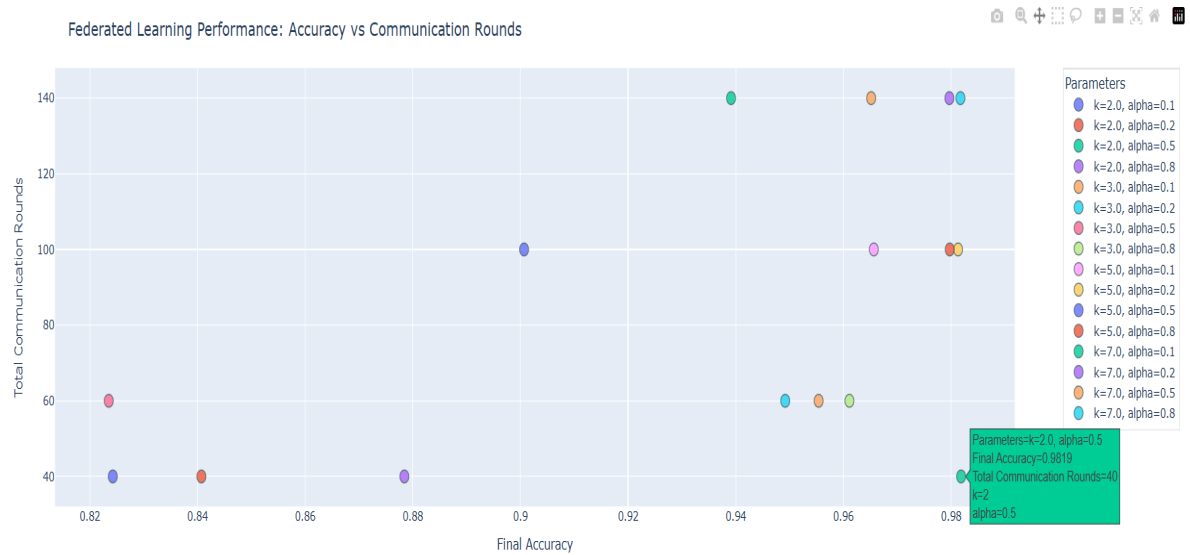


Figure 4.4: Accuracy versus communication rounds for MNIST under Non-IID conditions, where the highlighted point identifies the ideal balance of accuracy and communication efficiency.

Chapter 5: Conclusion and Future Works

This thesis addresses the critical challenge of client selection in federated learning environments, proposing and evaluating novel strategies to optimise accuracy and communication efficiency under varying data distributions. The thesis focused on static and dynamic client selection methods, including accuracy thresholds, K-values, and alpha parameters, with results highlighting the adaptability and efficiency of these approaches in both IID and non-IID settings.

Notably, the **Dynamic K and Alpha Selection method**, as detailed in Section 3.6.7, emerged as a robust strategy, **achieving a balance between high accuracy and reduced communication rounds**. Visualisations of accuracy versus communication rounds provided valuable insights into selecting ideal client configurations. However, the work was conducted in a simulated federated learning environment using datasets such as CIFAR-10 and MNIST, which, while effective for testing, may limit generalisability to more complex real-world scenarios.

The methodologies developed offer a foundation for improving client selection strategies in federated learning. However, challenges such as reliance on simulation-based testing, time-intensive experiments due to hardware constraints, and limited exploration of parameter tuning were identified. Addressing these will be crucial for broader applicability and further advancements in the field. Furthermore, this thesis holds potential for impact in domains such as healthcare and IoT systems, where effective client selection can drive improvements in distributed model training.

5.1 Future Work

Building on the insights and limitations of this thesis, several directions for future work are proposed:

Transition to Real-World Deployments

While this study relied on simulation-based environments, transitioning to real-world implementations is crucial for validating the practicality of the proposed strategies[16]. Evaluating methods on actual client nodes will provide insights into challenges such as communication latency, device availability, and security considerations.

Scaling to Larger and Diverse Datasets

Expanding the experimentation to larger and more diverse datasets beyond CIFAR-10 and MNIST will test the generalizability of the proposed methods. For example, applying these strategies to medical imaging or natural language processing tasks can evaluate scalability and robustness.

Exploration of Parameter Space

Extend experimentation by systematically exploring parameters such as accuracy thresholds, K-values, and alpha values as discussed in Section 3.6. Investigate the optimal number of participating clients per round to maximize efficiency and model performance.

Focus on Model Accuracy

While this thesis prioritised customised strategies, future work could emphasize designing and integrating high-performing models to establish a robust baseline for accuracy before applying advanced client selection methods.

Node Selection and Heterogeneity

Explore the impact of non-IID data on node selection strategies, emphasizing how client diversity influences global model performance. Consider heterogeneity in computational capacity among nodes to design methods that adapt to varying hardware capabilities, ensuring fairness and efficiency.

Advanced Visualisations and Decision-Making

Enhance the visualisation tools by integrating real-time updates, parameter impact analysis, and decision-support features to facilitate practical deployment.

Through these directions, future research can address current limitations, scale the methods for broader adoption, and enhance the practical viability of federated learning in diverse applications.

Bibliography

- [1] Get Started with Flower: <https://flower.ai/docs/framework/tutorial-series-get-started-with-flower-pytorch.html>.
- [2] Flwr-Datasets: <https://flower.ai/docs/datasets/>.
- [3] CNN: <https://www.tensorflow.org/tutorials/images/cnn>.
- [4] Google Colab: <https://colab.google/>.
- [5] Pytorch: <https://pytorch.org/>.
- [6] Torchvision: <https://pytorch.org/vision/stable/>.
- [7] Numpy: <https://numpy.org/doc/stable/>.
- [8] Matplotlib: <https://matplotlib.org/stable/>.
- [9] ReLU: <https://pytorch.org/docs/stable/generated/torch.nn.ReLU.html>.
- [10] Syreen Banabilah, Moayad Aloqaily, Eitaa Alsayed, Nida Malik, and Yaser Jararweh. Federated learning review: Fundamentals, enabling technologies, and future applications. *Information Processing Management*, 59(6):103061, 2022. URL: <https://www.sciencedirect.com/science/article/pii/S0306457322001649>, doi:10.1016/j.ipm.2022.103061.
- [11] Flower AI - What is Federated Learning?: <https://flower.ai/docs/framework/tutorial-series-what-is-federated-learning.html>.
- [12] M. I. Jordan and T. M. Mitchell. Machine learning: Trends, perspectives, and prospects. *Science*, 349(6245):255–260, 2015. doi:10.1126/science.aaa8415.
- [13] Jian Li, Tongbao Chen, and Shaohua Teng. A comprehensive survey on client selection strategies in federated learning. *Computer Networks*, 251:110663, 2024. URL: <https://www.sciencedirect.com/science/article/pii/S138912862400495X>, doi:10.1016/j.comnet.2024.110663.
- [14] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. Federated optimization in heterogeneous networks. *arXiv preprint arXiv:1812.06127*, 2020.
- [15] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. *arXiv preprint arXiv:1602.05629*, 2023.
- [16] Sarbaree Mishra, Vineela Komandla, Srikanth Bandi, Sairamesh Konidala, and Jeevan Manda. Training ai models on sensitive data - the federated learning approach. *Distributed Learning and Broad Applications in Scientific Research*, 6, Apr. 2020. URL: <https://dlabi.org/index.php/journal/article/view/250>.

- [17] Sashank Reddi, Zachary Charles, Manzil Zaheer, Zachary Garrett, Keith Rush, Jakub Konečný, Sanjiv Kumar, and H. Brendan McMahan. Adaptive federated optimization. *arXiv preprint arXiv:2003.00295*, 2021.
- [18] Hongyi Wang, Mikhail Yurochkin, Yuekai Sun, Dimitris Papailiopoulos, and Yasaman Khazaeni. Federated learning with matched averaging. *arXiv preprint arXiv:2002.06440*, 2020.
- [19] Jianyu Wang, Qinghua Liu, Hao Liang, Gauri Joshi, and H. Vincent Poor. Tackling the objective inconsistency problem in heterogeneous federated optimization. *arXiv preprint arXiv:2007.07481*, 2020.
- [20] Hongda Wu and Ping Wang. Probabilistic node selection for federated learning with heterogeneous data in mobile edge. *2022 IEEE Wireless Communications and Networking Conference (WCNC)*, pages 2453–2458, 2022. doi:10.1109/WCNC51071.2022.9771994.